

AI/ML Engineer Interview - RAG & GenAI

50 interview questions for a 5-year experience profile | Basic -> Intermediate -> Advanced -> Production

Use this as a preparation checklist before the mock interview. For each question, practice a 60-120 second answer, then add one real project example, one trade-off, and one production consideration. The "What the interviewer expects" line is a guide for self-review, not a model answer.

Interview area	What strong answers demonstrate
RAG fundamentals	Correct architecture, terminology, and clear reasoning.
Retrieval engineering	Chunking, hybrid search, metadata, reranking, query transformation.
Production AI engineering	Latency, cost, security, observability, evaluation, versioning.
Senior-level depth	Trade-offs, failure modes, measurable impact, and design decisions.

1. AI / ML and GenAI Foundations

1. What is the difference between traditional machine learning, deep learning, and generative AI?

What the interviewer expects: Define the scope of each and give practical examples. For GenAI, mention foundation models and generation of new content.

2. What is an LLM, and how is it different from a traditional NLP model?

What the interviewer expects: Explain token prediction, transformer architecture at a high level, pretraining, and why LLMs support many downstream tasks.

3. What are tokens, embeddings, context windows, and attention?

What the interviewer expects: Be able to connect these concepts: text -> tokens -> vectors, attention over context, and context-window limits.

4. What is a transformer, and why did it become dominant for language tasks?

What the interviewer expects: Explain self-attention, positional information, parallel training, and scaling benefits without going too deep into equations.

5. What is the difference between prompting, fine-tuning, and RAG?

What the interviewer expects: Explain when each is appropriate and why RAG is useful for private, fresh, or frequently changing knowledge.

6. What are hallucinations in LLMs, and why do they happen?

What the interviewer expects: Discuss missing knowledge, ambiguous prompts, weak retrieval, model uncertainty, and generation behavior. Mention grounding and evaluation.

7. What is temperature, and how does it affect LLM output?

What the interviewer expects: Know the relationship between temperature and randomness; explain why lower temperature is often preferred for factual or structured tasks.

8. What is the difference between zero-shot, few-shot, and chain-of-thought-style prompting?

What the interviewer expects: Explain examples in prompts, task specification, and the need to avoid exposing hidden reasoning. Focus on observable answer quality.

2. RAG Architecture and Design

9. Explain a complete RAG system end to end.

What the interviewer expects: Cover ingestion -> parsing -> chunking -> embedding -> indexing -> retrieval -> reranking -> prompt construction -> LLM generation -> citations/response.

10. Why would you choose RAG instead of fine-tuning for enterprise knowledge?

What the interviewer expects: Discuss freshness, private data, source traceability, lower retraining cost, and easier content updates.

11. What are the online and offline components of a RAG system?

What the interviewer expects: Offline: ingestion, cleaning, chunking, embeddings, indexing. Online: query processing, retrieval, reranking, generation, guardrails, logging.

12. How would you design a RAG system for millions of enterprise documents?

What the interviewer expects: Address distributed ingestion, metadata, partitioning, vector DB capacity, hybrid search, caching, access control, observability, and cost.

13. How would you implement multi-tenant RAG securely?

What the interviewer expects: Use tenant-aware metadata filters, isolated indexes/namespaces where needed, identity propagation, authorization checks, and leakage tests.

14. How would you build RAG over PDFs, Word files, web pages, and structured data?

What the interviewer expects: Discuss loaders, OCR where required, layout-aware parsing, tables, metadata normalization, and separate handling for structured sources.

15. How do you handle frequently changing documents in RAG?

What the interviewer expects: Use incremental ingestion, document versioning, deletion/upsert handling, freshness metadata, and re-embedding only changed content.

16. What is agentic RAG, and how is it different from basic RAG?

What the interviewer expects: Explain planning/tool use, iterative retrieval, query decomposition, multiple tool calls, and when agentic behavior is actually justified.

3. Chunking and Retrieval

17. What is chunking, and why is it important in RAG?

What the interviewer expects: Explain context preservation, retrieval granularity, embedding quality, and downstream prompt size.

18. How would you choose chunk size and overlap?

What the interviewer expects: Mention document type, semantic boundaries, model context, retrieval precision/recall, and empirical evaluation rather than one universal number.

19. What are semantic chunking and recursive chunking?

What the interviewer expects: Know how structural separators and semantic similarity can create more meaningful chunks than fixed character splits.

20. What is metadata filtering, and when should you use it?

What the interviewer expects: Use metadata for permissions, date, document type, team, region, product, version, and other high-signal constraints.

21. What is dense retrieval vs sparse retrieval?

What the interviewer expects: Dense uses embeddings for semantic similarity; sparse methods such as BM25 are strong for exact terms, IDs, names, and lexical matches.

22. What is hybrid search, and why is it useful?

What the interviewer expects: Combine dense and sparse retrieval. Explain score normalization/weighting or fusion methods and why enterprise queries benefit from both.

23. What is reranking, and why is it useful after initial retrieval?

What the interviewer expects: A reranker scores query-document pairs more deeply, improving relevance among top candidates at additional latency/cost.

24. What is query expansion or query rewriting in RAG?

What the interviewer expects: Transform ambiguous user queries into better retrieval queries; mention risks such as query drift and unnecessary latency.

25. How would you handle very short or ambiguous user queries?

What the interviewer expects: Use clarification, query rewriting, conversation context, metadata, fallback search, and confidence thresholds.

26. How do you reduce retrieval of irrelevant documents?

What the interviewer expects: Improve chunking, metadata filters, hybrid retrieval, reranking, top-k tuning, deduplication, and query rewriting.

4. Embeddings and Vector Databases

27. What is an embedding model?

What the interviewer expects: Explain mapping text into a vector space where semantic similarity can be measured.

28. How do you choose an embedding model for an enterprise RAG system?

What the interviewer expects: Consider domain quality, multilingual needs, dimensionality, latency, cost, hosting constraints, and evaluation on representative queries.

29. What is cosine similarity, and how is it used in vector search?

What the interviewer expects: Know the formula conceptually and why normalized vectors make cosine similarity useful for semantic retrieval.

30. Compare FAISS, Pinecone, Milvus, Weaviate, and pgvector at a high level.

What the interviewer expects: Discuss local/library use, managed service trade-offs, scale, filtering, operations, ecosystem, and existing database requirements.

31. What is an approximate nearest neighbor index?

What the interviewer expects: Explain why exact search is expensive at scale and how ANN methods trade small amounts of recall for speed.

32. How do you handle vector index updates and deletions?

What the interviewer expects: Discuss upserts, tombstones or delete operations, versioning, background compaction, and consistency.

33. How would you detect embedding drift or retrieval quality degradation?

What the interviewer expects: Monitor retrieval metrics, distributions, query types, model/version changes, and benchmark sets over time.

5. RAG Evaluation and Production Quality

34. How do you evaluate a RAG system?

What the interviewer expects: Separate retrieval evaluation from generation evaluation. Mention metrics such as recall@k, precision@k, MRR, faithfulness, answer relevance, and groundedness.

35. What is the difference between retrieval quality and answer quality?

What the interviewer expects: A system can retrieve the right document but generate a poor answer; or generate a plausible answer despite weak retrieval. Evaluate both layers.

36. What is RAGAS or similar RAG evaluation tooling used for?

What the interviewer expects: Explain automated evaluation of dimensions such as context relevance, context recall, faithfulness, and answer relevance, while noting judge-model limitations.

37. How would you create a golden dataset for RAG evaluation?

What the interviewer expects: Build representative questions, expected evidence/source IDs, acceptable answer criteria, edge cases, and adversarial examples.

38. How would you evaluate hallucination and groundedness?

What the interviewer expects: Require claims to be supported by retrieved evidence, compare response statements against source text, and track unsupported-claim rates.

39. How would you improve RAG latency in production?

What the interviewer expects: Use smaller/faster embedding and reranking models where acceptable, caching, parallel retrieval, fewer retrieved chunks, streaming, batching, and efficient indexes.

40. How do you reduce the cost of a RAG application?

What the interviewer expects: Control retrieval depth, use model routing, cache embeddings/results, compress context, use smaller models for simple steps, and monitor token usage.

41. What production metrics would you monitor for a RAG application?

What the interviewer expects: Latency, token usage, cost, retrieval hit rates, answer quality, citation quality, error rates, index freshness, cache hit rate, and user feedback.

6. Advanced AI Engineering / System Design

42. Design a production RAG assistant for an enterprise engineering team.

What the interviewer expects: Discuss architecture, document sources, Git/issue data, permissions, indexing, retrieval, citations, observability, evaluation, and deployment.

43. How would you secure a RAG system against prompt injection?

What the interviewer expects: Treat retrieved documents as untrusted data, separate instructions from content, validate tool calls, apply policies, and test malicious documents/prompts.

44. How would you prevent data leakage through RAG?

What the interviewer expects: Enforce authorization before retrieval, filter by user/role/tenant, avoid relying only on prompt instructions, and audit retrieved context.

45. How would you handle prompt injection stored inside a retrieved document?

What the interviewer expects: Detect or isolate suspicious instructions and ensure retrieved content cannot override system/developer policies or trigger unauthorized actions.

46. What is context compression, and when would you use it?

What the interviewer expects: Reduce retrieved content to the most useful facts to stay within context limits and reduce latency/token cost.

47. How would you design fallback strategies when retrieval returns low-confidence results?

What the interviewer expects: Use broader search, query rewriting, clarification, trusted external knowledge where allowed, or explicitly say there is insufficient evidence.

48. How do you version prompts, embeddings, indexes, and models in production?

What the interviewer expects: Treat each as a deployable artifact with metadata, evaluation gates, rollback capability, and compatibility checks.

49. How would you integrate RAG with tools or APIs to build an agent?

What the interviewer expects: Define tool schemas, permissions, planning/retry behavior, tool result validation, state handling, and strong guardrails.

50. Explain a difficult RAG problem you solved and how you measured the improvement.

What the interviewer expects: Give a concrete STAR-style story: problem, root cause, change, metrics before/after, trade-offs, and lessons learned.

Self-Assessment Before the Mock Interview

- Explain RAG end to end without looking at notes.
- Compare dense, sparse, hybrid retrieval and explain when each wins.
- Explain chunking choices and how you would evaluate them.
- Design a multi-tenant, secure RAG system.
- Explain how you would evaluate retrieval and generation separately.
- Discuss one RAG failure you would expect in production and how you would detect it.
- Explain how you would optimize latency and cost without hurting answer quality.
- Give a real project story with measurable before/after results.

Mock Interview Rule

During the next round, I will ask these questions one at a time. After every answer, I will give a match percentage, what you got right, what is missing, a corrected senior-level answer, and the next question.